ENGENHARIA E PROCESSOS DE SOFTWARE

Teste de Software

Prof. Manoel Dantas Macedo Filho

Introdução

O **teste de software** é um processo fundamental inserido no desenvolvimento de um software. Seu **objetivo principal é encontrar e revelar falhas e problemas** no software para que sejam corrigidos, garantindo que o produto final atinja a **qualidade desejada**. Qualidade, nesse contexto, significa que o software entregue atende aos requisitos e necessidades do cliente levantados no início do projeto.

Os profissionais da área de testes realizam um conjunto diversificado de atividades, que vão além dos testes funcionais da aplicação. Incluem também a avaliação da especificação de requisitos, avaliação de projeto técnico, verificações em outros documentos, testes de performance e capacidade, e avaliação da interface do sistema.

O Processo de Testes de Software

O teste de software é uma das atividades essenciais no processo de desenvolvimento de sistemas, com o diferencial de que **todas as suas ações são focadas exclusivamente em garantir a qualidade do software**.

O processo de teste básico é composto pelas seguintes atividades sequenciais e inter-relacionadas:

1. **Planejamento e Controle**.
2. **Análise e Modelagem**.
3. **Implementação e Execução**.
4. **Avaliação dos Critérios de Saída e Relatórios**.
5. **Atividades de Encerramento de Teste**.

Detalhes do Processo…

1. Planejamento e Controle:

- **É a primeira atividade a ser executada e serve como base para todo o processo de testes**.
- Sua finalidade principal é **estruturar e organizar** todas as partes e artefatos envolvidos nos testes.
- Define os **objetivos** e especifica as atividades do projeto de testes, além de estabelecer a forma de alcançá-los.
- Nesta etapa, é criado o **"plano de teste"**, um documento que sumariza todas as definições processuais, como: **pessoas envolvidas, tipos de testes, técnicas a serem usadas, riscos e artefatos necessários**.
- A atividade de **controle ocorre durante todo o processo**, verificando se o que foi definido no planejamento está sendo cumprido.

Detalhes do Processo…

2. Análise e Modelagem:

- Nesta etapa, são **criados os artefatos necessários** para cumprir o planejamento estabelecido.
- Exemplos de artefatos criados incluem **casos de teste e scripts de teste**.
- Também é verificada a **adequação do ambiente de testes**, realizando-se os ajustes necessários.
- Os casos de teste ou scripts de teste são **priorizados**, e é estabelecida uma **rastreabilidade bidirecional entre os requisitos e os casos de teste**.

3. Implementação e Execução:

- Nesta fase, os artefatos de teste que foram projetados são **organizados em suítes**, agrupados por prioridade ou por funcionalidade.
- Após a organização, essas suítes de testes são **executadas**.

Detalhes do Processo…

4. Avaliação dos Critérios de Saída e Relatórios:

- A execução dos testes é **avaliada com base nos objetivos** que foram definidos na etapa de planejamento.
- São gerados **relatórios** que indicam o progresso e os resultados dos testes.

5. Atividades de Encerramento de Teste:

- Nesta etapa final, **dados de todas as atividades de teste são coletados** para registrar a experiência adquirida, o "testware" (artefatos de teste), fatos e números relevantes.
- O encerramento ocorre em marcos importantes, como o **lançamento de um software**, a **conclusão ou cancelamento de um projeto de teste**, o **alcance de um marco de projeto** ou a **conclusão da implantação de uma demanda de manutenção**.

Níveis de Testes

Para a execução de um processo de testes completo, diversos níveis podem ser empregados, garantindo diferentes coberturas e objetivos. Os principais níveis são:

1. **Testes Unitários:**

    - Geralmente **executados pelo próprio programador** durante o desenvolvimento do software.
    - Focam no teste de **pequenas partes ou unidades isoladas do sistema**.
    - Uma técnica comum é o **Test Driven Development (TDD)**, onde o teste é criado antes da funcionalidade.
2. **Testes de Integração:**

    - Têm como objetivo **verificar se uma ou mais unidades "se encaixam perfeitamente"**.
    - Garantem que as peças que funcionam bem separadamente **se comuniquem e operem em pleno funcionamento quando unificadas**.

Níveis de Testes (Continuação)

3. **Testes de Sistema:**

    - É o tipo de teste **mais comumente empregado nas empresas de tecnologia**.
    - Visa a **testar o sistema ou uma funcionalidade como um todo**, em um ambiente que simula o uso real.
    - Pode ser realizado tanto de **forma manual quanto de forma automatizada**, utilizando diversas ferramentas.

4. **Testes de Aceitação:**

    - Este é o **último nível de testes** antes da entrega final do software.
    - É **altamente recomendado que seja feito com a presença do cliente ou de um representante** dele.
    - O **objetivo principal não é encontrar falhas**, mas sim **confirmar que tudo está implementado conforme foi solicitado**, visando a aceitação da entrega do produto pelo cliente.

Recursos Técnicos para Testes Unitários

Para facilitar a tarefa de testes de software, diversos recursos técnicos, como ferramentas, frameworks e linguagens de programação, podem ser utilizados.

Para **Testes Unitários**, a principal técnica é o **Test Driven Development (TDD)**. O TDD envolve três ciclos:

- **Red (Vermelho):** Escreve-se um teste que falha porque a funcionalidade ainda não foi programada.
- **Green (Verde):** Implementa-se a funcionalidade mínima para que o teste passe.
- **Refactor (Refatorar):** O código é aprimorado sem alterar sua funcionalidade, mantendo os testes passando.

Exemplos de recursos (frameworks) para aplicação de TDD, dependendo da linguagem de programação:

- **JUnit:** Utilizado para desenvolvimento em **Java**.
- **PHPUnit:** Utilizado para desenvolvimento em **PHP**.
- **Flask:** Utilizado para desenvolvimento em **Python**.

Recursos Técnicos para Testes de Sistema e Aceitação

Para **Testes de Sistema e Aceitação**, que compartilham o funcionamento de testar a ferramenta já desenvolvida como um todo, há uma **ampla gama de ferramentas** disponíveis.

Alguns exemplos notáveis incluem:

- **Sikuli:** Uma ferramenta de automação para **qualquer sistema com interface gráfica**, incluindo aplicações desktop.
- **Selenium WebDriver:** Uma ferramenta amplamente utilizada para **automação de testes em sistemas web**.
- **Robotium:** Uma ferramenta específica para **automação de testes de aplicativos móveis**.

Essas ferramentas permitem a criação de scripts que simulam a interação do usuário e verificam o comportamento do sistema de forma automatizada.

Testes de Caixa-Branca: Conceito e Características

- **Conceito:** O **teste de caixa-branca**, também conhecido como **teste estrutural** (ou "teste de caixa transparente" para alguns autores), é uma técnica que se concentra no **funcionamento interno do sistema**. Ele é aplicado nas **estruturas internas, no nível dos componentes do código-fonte**, onde as lógicas do programa já estão escritas, para verificar seu nível de qualidade.
- **Características Principais:**
  - Utiliza a **visão interna do sistema**, exigindo conhecimento sobre seu funcionamento detalhado.
  - Examina os **detalhes procedimentais e os caminhos lógicos do software**.
  - Os resultados fornecem informações **diretamente relacionadas ao código-fonte**, auxiliando as atividades de depuração e manutenção.
  - Embora os critérios possam ser usados em todas as fases, são **mais comuns em testes de unidade e testes de integração**.

Testes de Caixa-Branca: Conceito e Características (Continuação)

- **Principais Elementos/Critérios:**
  - **Notação de grafo de fluxos:** Ferramenta para representar o fluxo de controle lógico do programa.
  - **Complexidade Ciclomática:** Métrica que quantifica a complexidade da lógica de um programa, indicando o número de caminhos independentes e servindo para planejar a quantidade de testes.
  - **Caminho Básico:** Técnica para criar um conjunto base de caminhos de execução que garantam a cobertura do código.
  - **Caminhos Independentes:** Quaisquer caminhos que levam a um novo conjunto de comandos ou a uma nova condição.
  - **Teste de Estrutura de Controle:** Complementa o teste de caminho básico, ampliando a abrangência.
  - **Teste de Condição:** Exercita as condições lógicas de um módulo, avaliando resultados verdadeiros e falsos.
  - **Teste de Ciclo:** Foca na validação da construção de ciclos (simples, aninhados, concatenados, desestruturados).
  - **Teste de Fluxo de Dados:** Utiliza a análise do fluxo de dados para derivar casos de teste, baseando-se nas definições e usos das variáveis.

Testes de Caixa-Preta: Conceito e Características

- **Conceito:** O **teste de caixa-preta**, também conhecido como **teste funcional** ou **teste baseado em especificação**, é uma técnica que avalia o sistema a partir de suas **especificações externas**. Seu objetivo é garantir que os comportamentos resultantes das ações sejam **consistentes com os objetivos das especificações** dos requisitos.
- **Características Principais:**
  - **Não interessa como ocorre o funcionamento interno** ou a forma como a implementação do sistema foi feita. A análise é focada na entrada e saída, como se o sistema fosse uma "caixa-preta".
  - É **independente da linguagem de programação** ou do paradigma de desenvolvimento.
  - Pode ser **executado de modo manual ou com ferramentas de automação**.
  - É **mais comumente aplicado em nível de sistema**, mas pode ser usado em testes de unidade, integração e sistema.
  - É projetado com base em informações de especificação, como manuais do sistema, diagramas de caso de uso e documentos de requisitos funcionais/não funcionais.

- **Critérios de Aplicação:**
  Os testes de caixa-preta utilizam critérios específicos para determinar os casos de teste necessários, como: **partição de equivalência, análise de valor limite, tabela de decisão, grafo de causa e efeito, transição de estados e teste de caso de uso**.

CAIXA PRETA	CAIXA BRANCA
Verifica as operações e ações de uma aplicação.	Verifica o comportamento de uma aplicação.
É baseado nos requisitos do cliente.	É baseado nas expectativas do cliente.
Ajuda a melhorar o comportamento da aplicação.	Ajuda a melhorar o desempenho da aplicação.
O teste de caixa preta é fácil de executar manualmente.	É difícil executar o teste de caixa branca manualmente.
Ele testa o que o produto faz.	Ele descreve como o produto se sai.
O teste é baseado nos requisitos do negócio.	O teste é baseado nos requisitos de desempenho.
Exemplos	
1 - Teste de Unidade **2 - Teste de Integração** **3 - Teste de Regressão**	**1 - Teste de Desempenho** **2 - Teste de Carga** **3 - Teste de Estresse**

Curiosidades sobre Teste de Software

- O termo "caixa-branca" é, por vezes, considerado inadequado por alguns especialistas, como Schach (2008), que sugerem "caixa transparente" para melhor expressar a ideia de **visibilidade total do funcionamento interno** do sistema durante o teste.
- A métrica de **Complexidade Ciclomática**, uma ferramenta valiosa no teste de caixa-branca para prever módulos com alta probabilidade de erros, foi desenvolvida por **Thomas J. McCabe em 1976**, um profissional com experiência no Departamento de Defesa e Segurança Nacional dos EUA.
- O **Test Driven Development (TDD)** é uma técnica que **inverte a ordem tradicional**, onde **o teste é escrito antes mesmo da funcionalidade existir**. Isso não só garante a cobertura dos requisitos, mas também incentiva um design de código mais limpo e modular.
- A **automação de testes**, especialmente com ferramentas como Selenium WebDriver, permite que testes de caixa-preta (funcionais) sejam programados em diversas linguagens (como Java ou JavaScript) e executados automaticamente. Isso é **extremamente útil para testes de regressão**, garantindo que novas mudanças não causem problemas em funcionalidades existentes, através da execução de uma grande massa de testes de forma contínua e eficiente.

Obrigado!

[manoelmacedo@yahoo.com]

EXERCÍCIO

O **teste de caixa-preta**,

Pensando em uma planilha com os campos **“Nome, Idade, Experiência, Nota do teste, e RESULTADO”**, com este último tendo a fórmula: **=SE(E(B2>17;B2<65;C2>3;D2>7);"Aprovado";"Reprovado")**, fazer uma análise pensando nas formalizações dos tipos de teste **“Caixa-preta” e “Caixa-branca”, gerando um relatório simples, com suas conclusões obtidas, mesmo que sejam simples e mesmo que o contexto seja determinado pelas suas suposições.** A fórmula da planilha pode ser interpretada como um pequeno algoritmo lógico. Logo, podemos aplicar **teste da caixa-branca** verificando os **caminhos lógicos** e condições da fórmula.

1. Identificação dos critérios de decisão

A função `SE(E(...))` é uma expressão condicional com **quatro condições** que devem ser **todas verdadeiras** (por causa do `E` lógico):

- C1: `B2 > 17`
- C2: `B2 < 65`
- C3: `C2 > 3`
- C4: `D2 > 7`

2. Cobertura de caminhos lógicos

No teste da caixa-branca, você pode aplicar critérios como:

- **Cobertura de instruções**: testar se todas as instruções são executadas.
- **Cobertura de decisões**: testar todas as possibilidades de decisão (`Aprovado` ou `Reprovado`).
- **Cobertura de condições múltiplas**: testar todas as **combinações possíveis** das condições.

Com 4 condições booleanas, existem 24=162^4 = 1624=16 combinações possíveis de verdadeiros e falsos.

EXERCÍCIO

Você pode testar, por exemplo:

Caso de Teste	B2 (Idade)	C2 (Experiência)	D2 (Nota)	Resultado Esperado
CT1 (todos ok)	25	5	8	Aprovado
CT2 (idade baixa)	16	5	8	Reprovado
CT3 (idade alta)	70	5	8	Reprovado
CT4 (sem experiência)	25	2	8	Reprovado
CT5 (nota baixa)	25	5	6	Reprovado
CT6 (duas falhas)	16	2	8	Reprovado

Isso ajuda a **garantir que todos os caminhos lógicos da fórmula sejam validados**.

EXERCÍCIO

Conclusão da análise

No contexto da computação e do teste da caixa-branca:

- **A fórmula da planilha representa uma função com múltiplas condições compostas.**
- Você pode fazer uma **análise estrutural da lógica**, identificando todos os **caminhos possíveis** (com base em combinações de verdadeiro/falso).
- Testes devem cobrir todos esses caminhos para garantir a **correta execução da lógica**.

Inclusive, a planilha pode ser usada como exemplo didático para ensinar:

- Como funções condicionais simulam decisões em código;
- Como aplicar critérios de cobertura de teste;
- Como projetar casos de teste baseados na estrutura lógica.

✅ Tabela com os 16 casos:

Caso	C1 (B2>17)	C2 (B2<65)	C3 (C2>3)	C4 (D2>7)	Esperado	Exemplo: Idade	Exemplo: Exp	Exemplo: Nota
CT01	V	V	V	V	Aprovado	30	5	8
CT02	V	V	V	F	Reprovado	30	5	7
CT03	V	V	F	V	Reprovado	30	2	8
CT04	V	V	F	F	Reprovado	30	2	7
CT05	V	F	V	V	Reprovado	70	5	8
CT06	V	F	V	F	Reprovado	70	5	6
CT07	V	F	F	V	Reprovado	70	2	8
CT08	V	F	F	F	Reprovado	70	2	6
CT09	F	V	V	V	Reprovado	16	5	8
CT10	F	V	V	F	Reprovado	16	5	6
CT11	F	V	F	V	Reprovado	16	2	8
CT12	F	V	F	F	Reprovado	16	2	6
CT13	F	F	V	V	Reprovado	70	5	8
CT14	F	F	V	F	Reprovado	70	5	6
CT15	F	F	F	V	Reprovado	70	2	8
CT16	F	F	F	F	Reprovado	70	2	6

Finalmente, dentro da abordagem de teste da caixa-branca você pode (e deve) sugerir melhorias na lógica, especialmente quando: 1) A regra atual é **muito rígida ou limitada**; 2) Existem **exceções relevantes que a lógica ignora**; e 3) O objetivo é tornar o sistema mais **realista, justo e eficaz**. Pensar em “Pesos” pode ajudar uma empresa, por exemplo, a verificar que um candidato com mais de 65 anos pode ser uma boa opção se ele tiver muita experiência e uma boa nota no teste de admissão, sendo um exemplo clássico de como bons testes podem levar a **um sistema mais inteligente**, e não só "correto" tecnicamente.